ID2230 Data Structures and Applications — Midterm
Version 1 Duration: 50 minutes Total: 80 marks

Instructions. Answer all questions. Clearly mark one correct answer out of the options provided. Any ambiguous markings will get zero credit.

Conventions. Use tight asymptotic bounds. Arrays have spare capacity for insertion; storage allocation takes constant time. BSTs are unbalanced and contain distinct keys. For recurrences, assume $T(1) = \Theta(1)$ and suitable integer sizes; logarithm bases do not affect asymptotic bounds. Each BST construction question starts with a fresh tree.

Section A — Multiple Choice

Circle exactly one answer per question. 2 marks each.

1. Which BST node has no successor? [2]

   (a) Every leaf node.
   (b) The root of every BST.
   (c) The maximum-key node.
   (d) The minimum-key node.

2. The worst-case time to merge two sorted arrays of lengths p and q into one sorted array is: [2]

   (a) $\Theta(p + q)$
   (b) $\Theta(pq)$
   (c) $\Theta(\log(p + q))$
   (d) $\Theta((p + q)^2)$

3. Five customers arrive, at times (minutes) $0, 2, 3, 3, 7$, and are enqueued in that order. Each requires exactly 4 minutes of service from a single server; service of the next customer begins the moment the previous one finishes, or the moment that customer arrives, whichever is later. At what time does the last (5th) customer's service finish? [2]

   (a) 20
   (b) 19
   (c) 24
   (d) 16

4. Consider the code below.

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < 5; j++)
        doSomething();
```

   For $n = 20$, how many total times is `doSomething()` called, and what is time complexity of the code snippet? [2]

   (a) count $= 100; O(n)$

(b) count $= 20; O(n)$

(c) count $= 100; O(n^2)$

(d) count $= 400; O(n^2)$

5. Binary search searches for 9 in $[2, 5, 8, 12, 16]$, using inclusive zero-based bounds and the floor of their average as midpoint. Which keys are examined? [2]

   (a) $12, 8$

   (b) $2, 5, 8, 12$

   (c) $8, 12$

   (d) $8, 5, 12$

6. Why does the usual $\Theta(\log n)$ array binary-search bound not directly carry over to an ordinary singly linked list? [2]

   (a) A linked list cannot store sorted keys.

   (b) Accessing a midpoint requires traversing links.

   (c) Linked-list comparisons take linear time each.

   (d) Binary search requires all keys to be distinct.

7. The values $1, 2, 3, 4$ must be pushed onto a stack in that exact order (pops may be interleaved at any point). Which of the following output sequences is impossible to produce this way? [2]

   (a) $2, 1, 4, 3$

   (b) $2, 4, 3, 1$

   (c) $3, 1, 4, 2$

   (d) $4, 3, 2, 1$

8. A circular queue has `CAPACITY`=6 and tracks size with a separate count variable. Rules: `enqueue(x)` stores x at `arr[rear]`, then sets `rear=(rear+1)%CAPACITY`; `dequeue()` reads `arr[front]`, then sets `front=(front+1)%CAPACITY`. Starting from `front=rear=0` (empty): enqueue 3 elements, dequeue 1, then enqueue 4 more. What are the final values of `front`, `rear`, and `count`? [2]

   (a) front $= 1,$ rear $= 1,$ count $= 7$

   (b) front $= 0,$ rear $= 1,$ count $= 6$

   (c) front $= 1,$ rear $= 1,$ count $= 6$

   (d) front $= 1,$ rear $= 0,$ count $= 6$

9. For a BST of height h edges, the tight worst-case search time as a function of h is: [2]

   (a) $\Theta(2^h)$

   (b) $\Theta(h)$

   (c) $\Theta(1)$

   (d) $\Theta(\log(h))$

10. You are given a direct pointer to node X in the middle of a singly linked list (X is not the last node), with no access to the list's head or any "previous" pointer. Through which of the following approaches you can effectively delete X in $O(1)$ time? [2]

(a) Traverse from X forward to find X again, then delete it

(b) Simply set X's data field to 0

(c) It cannot be done without the head pointer

(d) Copy the data from X's next node into X, then delete/free that next node instead, updating X's next to skip over it

11. A plain BST is built by inserting distinct keys $1, 2, \ldots, n$ in this order. Its height in edges is: [2]

(a) $\lfloor \log_2 n \rfloor$

(b) $n - 1$

(c) n^2

(d) 1

12. An algorithm with time complexity $O(n^3)$ takes 2 milliseconds to run on an input of size $n = 5$. Approximately how long will it take on an input of size $n = 20$ (assuming the constant factor dominates and stays fixed)? [2]

(a) 64 ms

(b) 32 ms

(c) 256 ms

(d) 128 ms

13. An element at a known index is deleted from an unsorted array. Order need not be preserved. What is the best worst-case time? [2]

(a) $\Theta(1)$

(b) $\Theta(n)$

(c) $\Theta(\log n)$

(d) $\Theta(n \log n)$

14. What is the tight bound for $T(n) = 9T(n/3) + n^2$? [2]

(a) $\Theta(n^3)$

(b) $\Theta(n^2)$

(c) $\Theta(n^2 \log n)$

(d) $\Theta(n \log n)$

15. Bubble sort processes $[2, 3, 4, 1]$ in ascending order with left-to-right passes. After the first complete pass, where is key 1 (assume that the first element of the array is at index 1)? [2]

(a) 3

(b) 1

(c) 4

(d) 2

16. A sorted singly linked list is searched from its head to delete a node with a supplied key. What is the worst-case time, including locating the node? [2]

(a) $\Theta(n \log n)$

(b) $\Theta(n)$

(c) $\Theta(1)$

(d) $\Theta(\log n)$

17. What is the tight bound for $T(n) = 2T(n/4) + 1$? [2]

(a) $\Theta(n \log n)$

(b) $\Theta(n)$

(c) $\Theta(\log n)$

(d) $\Theta(\sqrt{n})$

18. A sorted ascending array has spare capacity to its right. A new key is known to be larger than every existing key. What is the time to insert it while preserving sorted order? [2]

(a) $\Theta(n)$

(b) $\Theta(\log n)$

(c) $\Theta(n^2)$

(d) $\Theta(1)$

19. A stack starts empty. A total of exactly 6 push/pop operations occur (in some order, always valid), and the final stack contains exactly 2 elements. How many of the 6 operations were pops? [2]

(a) 1

(b) 2

(c) 4

(d) 3

20. When deleting a BST node with two children, which replacement always preserves the BST ordering after the replacement key is removed from its original location? [2]

(a) Its parent, regardless of position.

(b) Any leaf in its left subtree.

(c) Its successor.

(d) The maximum key in its right subtree.

21. A naive queue uses a plain (non-circular) array; each `dequeue()` removes `arr[0]` and shifts all remaining elements one position left. We enqueue 7 elements, then call `dequeue()` seven times in a row. What is the total number of shift operations performed, and the time complexity of draining n elements this way? [2]

(a) $28; O(n^2)$

(b) $21; O(n)$

(c) $42; O(n^2)$

(d) $21; O(n^2)$

22. Which of the following has $O(1)$ auxiliary (extra) space complexity, regardless of the input size n? [2]

(a) An algorithm that uses a fixed, small number of extra scalar variables no matter how large n is

(b) A recursive algorithm whose recursion depth grows with n

(c) An algorithm that builds an $n \times n$ 2-D array

(d) An algorithm that allocates a new array of size n

23. Consider the code below, meant to delete the last node from a list of 2 or more nodes.

```
struct Node* headRef;
void deletelast() {
    struct Node* temp = headRef;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}
```

When does this function crash or misbehave? [2]

(a) Neither case causes a problem

(b) Only when the list has exactly one node; an empty list is fine

(c) Both when the list is empty and when it has exactly one node

(d) Only when the list is empty; a single-node list is fine

24. A function counts the nodes in a 9-node singly linked list by starting a counter at 0 and a pointer at head, incrementing the counter and advancing the pointer once per node visited, stopping when the pointer becomes `NULL`. How many times does the pointer get advanced? [2]

(a) 10

(b) 0

(c) 9

(d) 8

25. Inserting a new key into a sorted array while keeping it sorted takes what worst-case time, even with binary search for the position? [2]

(a) $\Theta(n)$

(b) $\Theta(1)$

(c) $\Theta(n \log n)$

(d) $\Theta(\log n)$

26. A function prints a linked list using `while(temp!=NULL){...; temp=temp->next;}`, but is accidentally given a circular linked list (the last node's next points back to the head, instead of `NULL`). What happens? [2]

(a) It runs forever, repeatedly printing the same value sequence

(b) It prints all nodes once and terminates correctly

(c) It crashes immediately

(d) It prints nothing at all

27. Which recurrence is outside the standard Master Theorem form? [2]

    (a) $T(n) = 3T(n/2) + n$

    (b) $T(n) = T(n-1) + n$

    (c) $T(n) = T(n/2) + 1$

    (d) $T(n) = 4T(n/2) + n^3$

28. Consider the code below.

```
int count = 0;
for (int i = 1; i <= n; i *= 3) count++;
```

    For $n = 100$, what is the final value of `count`, and what is the time complexity of the code using Big-O notation? [2]

    (a) count $= 5; O(n)$

    (b) count $= 5; O(\log n)$

    (c) count $= 4; O(\log n)$

    (d) count $= 7; O(\log n)$

29. Consider the following operation sequences:

    $S1$ (Sequence of operations on a queue): `enqueue(A); enqueue(B); dequeue(); enqueue(C); enqueue(D); dequeue(); enqueue(E); dequeue(); enqueue(F);`

    $S2$ (Sequence of operations on a queue): `enqueue(A); enqueue(B); enqueue(C); dequeue(); dequeue(); enqueue(D); enqueue(E); dequeue(); enqueue(F);`

    $S3$ (Sequence of operations on a stack): `push(A); push(B); pop(); push(C); push(D); pop(); push(E); pop(); push(F);`

    $S4$ (Sequence of operations on a stack): `push(A); push(B); push(C); pop(); pop(); pop(); push(D); push(E); push(F);`

    Which of the sequences do not result in the same final configurations? [2]

    (a) $S1$ and $S4$

    (b) $S2$ and $S3$

    (c) $S2$ and $S4$

    (d) $S1$ and $S2$

30. Quicksort partitions $[9, 3, 7, 1, 6]$ around pivot 6 and places it in position. What is the index of 6 after the partition? Assume the indexing of the array starts with the index 1. [2]

    (a) 3

    (b) 4

    (c) 1

    (d) 2

31. Consider the code below.

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n - i + 1; j++)
        work();
```

For $n = 6$, how many total times is `work()` called, and what is the time complexity of the code? [2]

(a) count $= 15; O(n^2)$

(b) count $= 36; O(n^2)$

(c) count $= 21; O(n)$

(d) count $= 21; O(n^2)$

32. Which statement about randomized quicksort is correct? [2]

(a) Each pivot is guaranteed to be the median.

(b) Every input is sorted in worst-case $\Theta(n \log n)$ time.

(c) Randomization eliminates recursive calls.

(d) Its worst-case time remains $\Theta(n^2)$.

33. For binary search in an ascending array, suppose we are searching for x. If $A[\text{mid}] < x$, the next inclusive search interval should be: [2]

(a) $[\text{mid}, \text{high}]$

(b) $[\text{low}, \text{mid}]$

(c) $[\text{low}, \text{mid} - 1]$

(d) $[\text{mid} + 1, \text{high}]$

34. Consider the code below.

```
#define MAX 3
int arr[MAX], top = -1;
void push(int x) {
    if (top == MAX - 1) { printf("Overflow\n"); return; }
    arr[++top] = x;
}
int pop() { return arr[top--]; }
```

We call: `push(5), push(15), push(25), push(35), pop(), push(45)`. On which call(s) does "Overflow" print (and how many times), and what is the final value of `top`? [2]

(a) Overflow once, on push(35); final top=2, holding 45

(b) Overflow once, on push(25); final top=1, holding 35

(c) Overflow twice; final top=2, holding 45

(d) No overflow occurs; final top=3, holding 45

35. A quicksort call on n distinct keys chooses its pivot from the last position of the array. If k keys are smaller than the pivot, the two recursive subproblem sizes are: [2]

(a) k and $n - k - 1$

(b) $k-1$ and $n-k$

(c) $k+1$ and $n-k-1$

(d) k and $n-k$

36. Consider the code below.

```
void f(int n) {
    if (n < 1) return;
    f(n / 2);
}
```

Counting the initial call, how many total calls to `f` occur for `f(20)` (using integer division), and what is the code's time complexity? [2]

(a) calls $= 5; O(\log n)$

(b) calls $= 20; O(\log n)$

(c) calls $= 6; O(\log n)$

(d) calls $= 6; O(n)$

37. Consider the code below.

```
int f(int n) {
    if (n <= 1) return 1;
    return f(n - 1) + f(n - 1) + f(n - 1);
}
```

Counting the initial call, how many total calls to `f` occur for `f(3)`, and what is the code's time complexity? [2]

(a) calls $= 27; O(3^n)$

(b) calls $= 9; O(3^n)$

(c) calls $= 13; O(2^n)$

(d) calls $= 13; O(3^n)$

38. Given a pointer to a non-tail node p in a singly linked list, deleting the node immediately after p takes: [2]

(a) $\Theta(1)$

(b) $\Theta(n)$

(c) $\Theta(n^2)$

(d) $\Theta(\log n)$

Section B — Draw Type

Write your answer in the space provided. 4 marks.

(1.) Insert $45, 25, 65, 15, 35, 55, 75, 10, 32, 58$, in that order, into an initially empty binary search tree. Draw the BST after each insertion, producing 10 figures. [4]

(Questions are apparently same but the order changes corresponding to Version)